COMP2007: Operating Systems & Concurrency
Week 8 – 12:00pm Monday – 11 November 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

Operating Systems and Concurrency

Memory Management 4

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

Goals for Today

Overview

- **Address translation** implementation
- **Virtual memory** and **page faults**
- **Multi-level page tables**
- **Translation Look Aside Buffers** (TLBs)

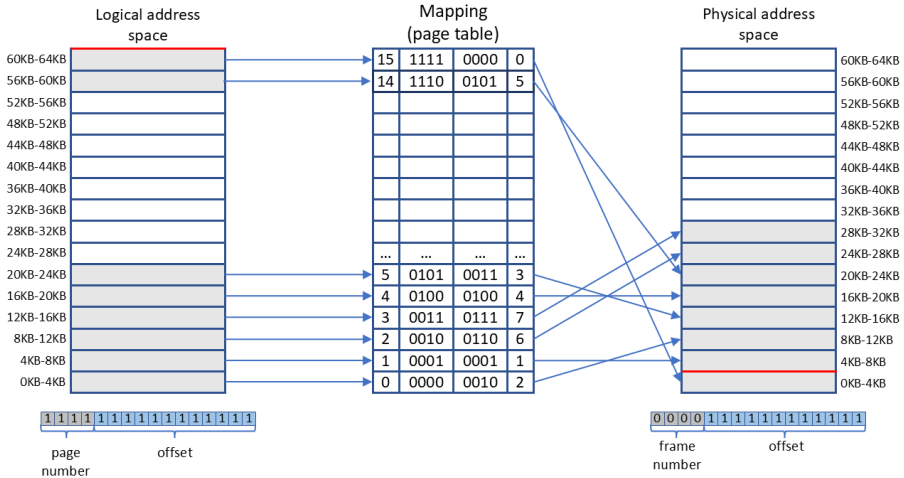
Recall

Last Lecture

- The **principles of paging** are:
  - **Logical address space** is divided into equal sized **pages**
  - **Physical address space** is divided into equal sized **frames**
  - **Non-contiguous** memory management scheme (although pages themselves are contiguous)
  - A **page table** contains multiple "**relocation registers**" to map the pages on to frames
  - The **offset** within the page and frame **remains the same** (they are the same size)
- The **benefits** of paging already discussed include:
  - Reduced **internal fragmentation**
  - No **external fragmentation**

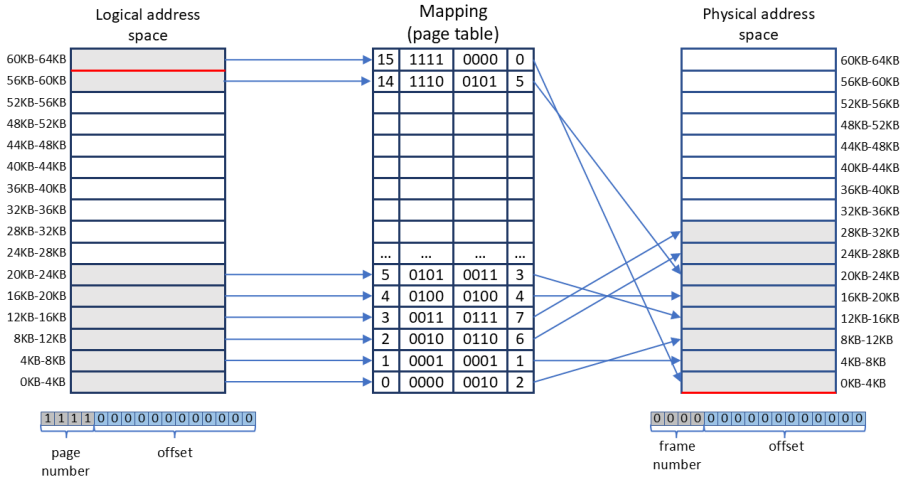
Paging

Address Translation: Implementation



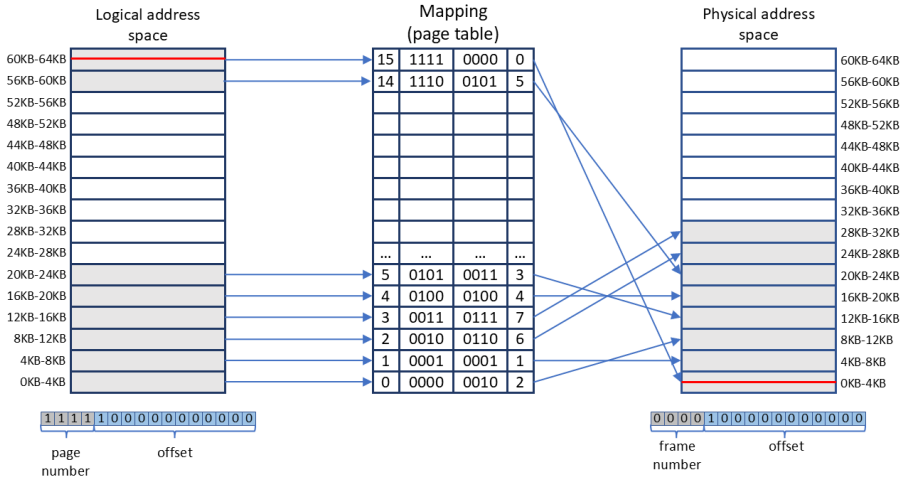
Paging

Address Translation: Implementation



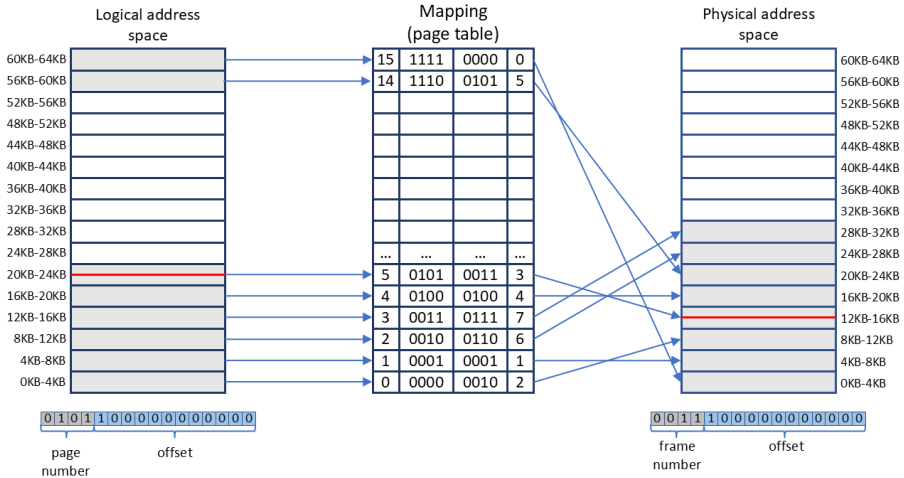
Paging

Address Translation: Implementation



Paging

Address Translation: Implementation



Paging

Relocation: Address Translation

- Steps in **address translation**:
  1. **Extract the page number** from logical address
  2. Use page number as an index to **retrieve the frame number** in the page table
  3. **Substitute** the **page number** with the **frame number**
- **Hardware implementation** of address translation
  1. The CPU's **memory management unit** (MMU) intercepts logical addresses
  2. MMU uses a page table as above
  3. The resulting **physical address** is put on the **memory bus**

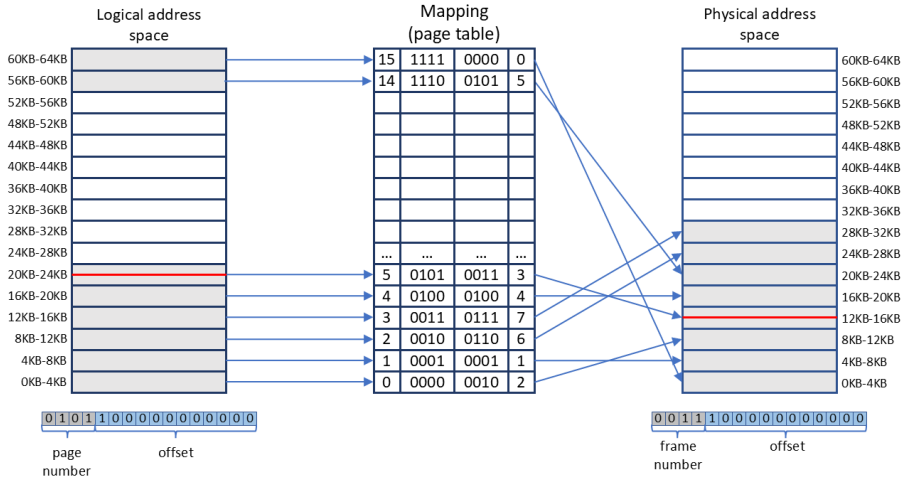
Virtual Memory

Principle of Locality

- Are there any **other benefits of paging**?
  - Principle of **locality**: **code** and **data references** are usually **clustered**
  - Code execution and data manipulation is usually restricted to a small **subset of pages** point in time
- **Not all pages** have to be **loaded** in memory at the same time ⇒ **virtual memory**
  - Loading all program/data pages into memory is **wasteful**
  - Desired blocks could be **loaded on demand**

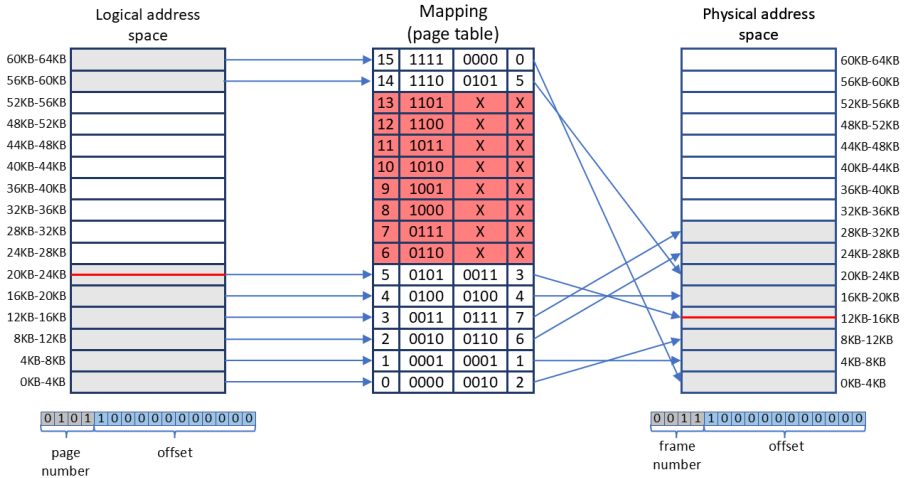
Virtual Memory

An Example



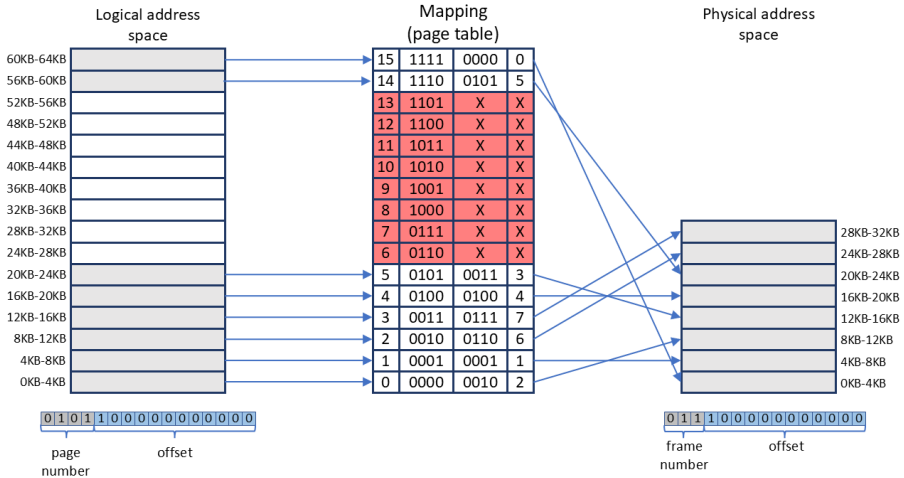
Virtual Memory

An Example



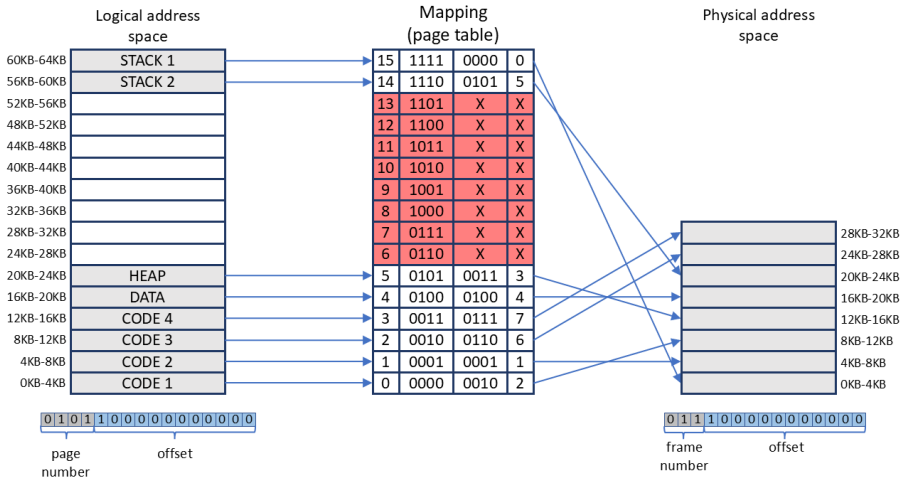
Virtual Memory

An Example



Virtual Memory

An Example



Virtual Memory

Page Faults

- The **resident set** refers to the pages that are loaded in main memory
- A **page fault** is generated if the processor accesses a page that is **not in memory**
  - A page fault results in an **interrupt** (process enters **blocked state**)
  - An **I/O operation** is started to bring the missing page into main memory
  - A **context switch** (may) take place
  - An **interrupt signals** that the I/O operation is complete (process enters **ready state**)

Demand Paging

Processing Page Faults

```
1. Trap operating system
  - Save registers/process state
  - Analyse interrupt (i.e., identify page fault)
  - Validate page reference, determine frame location
  - Issue disk I/O: queueing, seek, latency, transfer
2. Context switch (optional)
3. Interrupt for I/O completion
  - Store process state/registers
  - Analyse interrupt from disk
  - Update page table (page now in memory)
  - Wait for original process to be scheduled
4. Context switch to original process
```

Virtual Memory

What are The Benefits

1. Virtual memory **improves CPU utilisation**
   - Individual **processes take up less memory** (only partially loaded) $\Rightarrow$ **more processes** in memory
   - More **efficient use of memory** (less internal fragmentation, no external fragmentation)
2. The **logical address space** can be **larger than physical address space**
   - 64 bit machine $\Rightarrow$ 2^{64} logical addresses (theoretically)

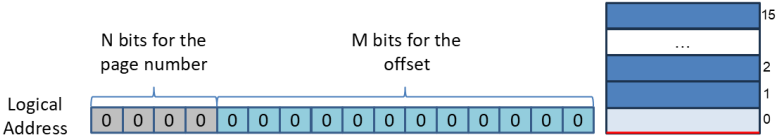


Figure: Logical Address

Virtual Memory

What are The Benefits

1. Virtual memory **improves CPU utilisation**
   - Individual **processes take up less memory** (only partially loaded) ⇒ **more** [illegible]s in memory
   - More **efficient use of memory** (less internal fragmentation, no external fra[illegible]
2. The **logical address space** can be **larger than physical address space**
   - 64 bit machine ⇒ 2^{64} logical addresses (theoretically)

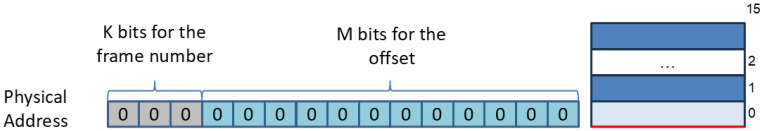


Figure: Logical Address

Virtual Memory

Page Tables Revisited: Contents of a Page Entry

- The **"present/absent bit"** is set if the **page/frame is in memory**
- The **"modified bit"** is set if the **page/frame has been modified**
  - Only modified pages have to be written back to disk when evicted
- The **"referenced bit"** that is set if the page is in use
- **Protection and sharing bits**: read, write, execute or combinations thereof

Other Info	Referenced	Modified	Protection	Present/Absent	
...	0/1	0/1	RWX	0/1	Frame Number

Figure: Page table entry

Virtual Memory

Page Tables Revisited: Page Table Size

- For a **16 bit machine**, the total address space is 2^{16}
  - Assuming that 10 bits are used for the offset (2^{10})
  - 6 bits can be used to number the pages
  - I.e., 2^6 = 64 pages can be maintained

Virtual Memory

Page Tables Revisited: Page Table Size

- For a **16 bit machine**, the total address space is 2^{16}
  - Assuming that 10 bits are used for the offset (2^{10})
  - 6 bits can be used to number the pages
  - I.e., 2^6 = 64 pages can be maintained
- For a **32 bit machine**, total address space is 2^{32}
  - Assuming pages of 2^{12} bytes (4KB)
  - 20 bits can be used to number the pages
  - I.e., 2^{20} pages (approx. 1 million) can be maintained
  - 4MB at 4 bytes per page table entry!

Virtual Memory

Page Tables Revisited: Page Table Size

- For a **16 bit machine**, the total address space is 2^{16}
  - Assuming that 10 bits are used for the offset (2^{10})
  - 6 bits can be used to number the pages
  - I.e., 2^6 = 64 pages can be maintained
- For a **32 bit machine**, total address space is 2^{32}
  - Assuming pages of 2^{12} bytes (4KB)
  - 20 bits can be used to number the pages
  - I.e., 2^{20} pages (approx. 1 million) can be maintained
  - 4MB at 4 bytes per page table entry!
- For a **64 bit machine** . . .

1. **Size**: how do we deal with **the increasing size of page tables** and **where do we store** them?
   - Their size prevents them from being **stored in registers**
   - Increasing the **page size** reduces the **page table size** (e.g, $n + m = 32$) but increases fragmentation
2. **Speed**: address translation happens at every memory reference, it has to be fast!
   - Accessing main memory results in **memory stalls**
   - How can we maintain **size manageable** and **speeds acceptable**?

Virtual Memory

Page Tables Revisited: Multi-level Page Tables

- **Solution**:
  - Use **multi-level page tables** (**tree-like** structures)
  - Store the page table in **(virtual) main memory** (page the page table!)
- The **page number** is divided into:
  - An index into the root page table
  - An index into the **second level page table** containing the **page to frame mapping**
- The page table is **not kept entirely in memory**

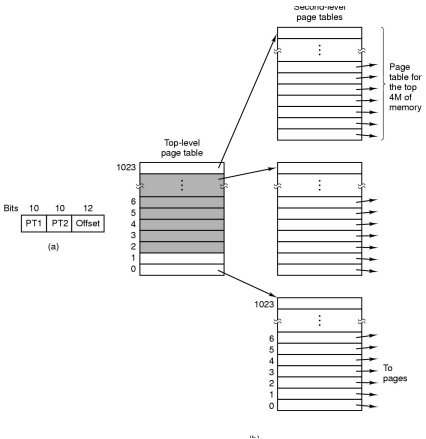


Figure: Multi-level page tables (from Tanenbaum)

Virtual Memory

Page Tables Revisited: Multi-level Page Tables

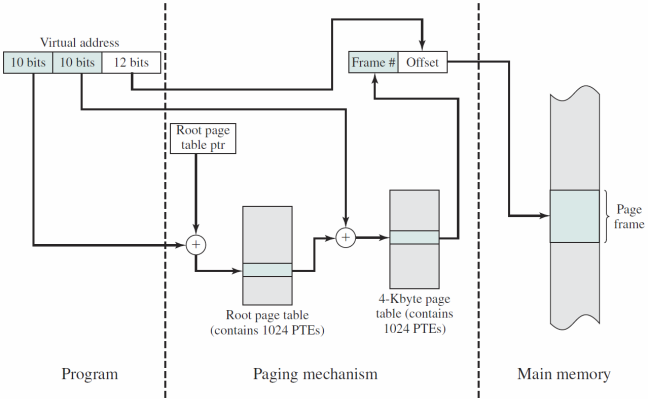


Figure: Multi-level Address Translation (from Stallings)

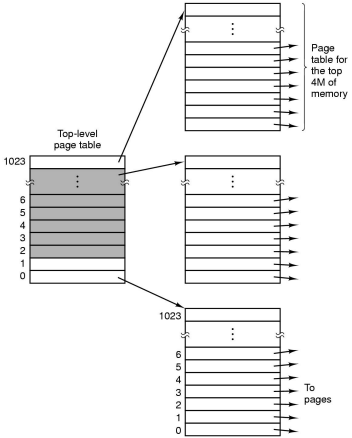


Figure: Multi-level page tables (from Tanenbaum)

©University of Nottingham

Virtual Memory

Access Speed on [multi-level] Page Tables

- **Memory organisation** of multi-level page tables:
  - The **root page table** is always maintained in memory
  - Page tables themselves are maintained in **virtual memory** due to their size
- Assume that a **fetch** from main memory takes T nano seconds
  - With a **single page table level**, access is $2 \times T$
  - With **two page table levels**, access is $3 \times T$
  - ...

Virtual Memory

Access Speed on [multi-level] Page Tables (Cont'ed)

- With two levels, every memory reference already becomes **3 times slower**:
  - Memory access already forms a **bottleneck** under normal circumstances
  - This assumes that the second level page table is **already in main memory**
- Multi-level page tables contain **up to 5 levels** in practice

Virtual Memory

Access Speed on [multi-level] Page Tables (Cont'ed)

- With two levels, every memory reference already becomes **3 times slower**:
  - Memory access already forms a **bottleneck** under normal circumstances
  - This assumes that the second level page table is **already in main memory**
- Multi-level page tables contain **up to 5 levels** in practice

Address Translation

How do we maintain **speed** of address translations?

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- **Translation look aside buffers** (TLBs) are (usually) located inside the memory management unit
  - They **cache** the most frequently used page table entries
  - They can be searched **in parallel**
- The principle behind TLBs is similar to other types of **caching in operating systems**
- Remember: **locality** states that processes make a **large number of references** to a **small number of pages**

Virtual Memory

Translation Look Aside Buffers (TLBs)

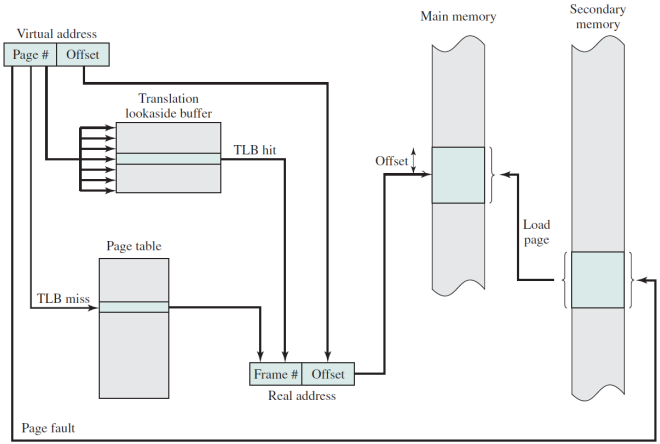


Figure: TLB Address Translation with a single-level page table(from Stallings)

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- Memory access with TLBs:
  - Assume a single-level page table
  - Assume a 20ns associative **TLB lookup**
  - Assume a 100ns **memory access time**
    - What is the access time for a **TLB hit**:

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- Memory access with TLBs:
  - Assume a single-level page table
  - Assume a 20ns associative **TLB lookup**
  - Assume a 100ns **memory access time**
    - What is the access time for a **TLB hit**: 20 + 100 = 120 ns
    - What is the access time for a **TLB miss**:

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- Memory access with TLBs:
  - Assume a single-level page table
  - Assume a 20ns associative **TLB lookup**
  - Assume a 100ns **memory access time**
    - What is the access time for a **TLB hit**: 20 + 100 = 120 ns
    - What is the access time for a **TLB miss**: 20 + 100 + 100 = 220ns
- Performance evaluation of TLBs:
  - What is the access time for an 80% hit rate?

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- Memory access with TLBs:
  - Assume a single-level page table
  - Assume a 20ns associative **TLB lookup**
  - Assume a 100ns **memory access time**
    - What is the access time for a **TLB hit**: 20 + 100 = 120 ns
    - What is the access time for a **TLB miss**: 20 + 100 + 100 = 220ns
- Performance evaluation of TLBs:
  - What is the access time for an 80% hit rate?
    - $120 \times 0.8 + 220 \times (1 - 0.8) =$ 140ns (i.e. 40% slowdown – relative to absolute addressing)

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- Memory access with TLBs:
  - Assume a single-level page table
  - Assume a 20ns associative **TLB lookup**
  - Assume a 100ns **memory access time**
    - What is the access time for a **TLB hit**: 20 + 100 = 120 ns
    - What is the access time for a **TLB miss**: 20 + 100 + 100 = 220ns
- Performance evaluation of TLBs:
  - What is the access time for an 80% hit rate?
    - $120 \times 0.8 + 220 \times (1 - 0.8) = 140$ns (i.e. 40% slowdown – relative to absolute addressing)
  - What is the access time for an 98% hit rate?

Virtual Memory

Page Tables Revisited: Translation Look Aside Buffers (TLBs)

- Memory access with TLBs:
  - Assume a single-level page table
  - Assume a 20ns associative **TLB lookup**
  - Assume a 100ns **memory access time**
    - What is the access time for a **TLB hit**: 20 + 100 = 120 ns
    - What is the access time for a **TLB miss**: 20 + 100 + 100 = 220ns
- Performance evaluation of TLBs:
  - What is the access time for an 80% hit rate?
    - $120 \times 0.8 + 220 \times (1 - 0.8) = 140$ns (i.e. 40% slowdown – relative to absolute addressing)
  - What is the access time for an 98% hit rate?
    - $120 \times 0.98 + 220 \times (1 - 0.98) = 122$ns (i.e. 22% slowdown)
- Note that **page tables** can be **held in virtual memory** $\Rightarrow$ further (initial) slow down due to page faults

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame:

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame: 12 (2^{12} addresses)
  - How many bits are used for the page number:

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame: 12 (2^{12} addresses)
  - How many bits are used for the page number: 4 (2^4 pages)
  - How many bits are used for the frame number:

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame: 12 (2^{12} addresses)
  - How many bits are used for the page number: 4 (2^4 pages)
  - How many bits are used for the frame number: not specified

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0:

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0:
  - Page index 0 offset 0 $\Rightarrow$ frame index 2 offset 0
  - 2 * 4096 = 8192

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0:
  - Page index 0 offset 0 $\Rightarrow$ frame index 2 offset 0
  - 2 * 4096 = 8192
- What is the physical address for logical address 8192:
  - Page index 2 offset 0 $\Rightarrow$ frame with index 6 offset 0
  - 6 * 4096 = 24576

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0:
  - Page index 0 offset 0 $\Rightarrow$ frame index 2 offset 0
  - 2 * 4096 = 8192
- What is the physical address for logical address 8192:
  - Page index 2 offset 0 $\Rightarrow$ frame with index 6 offset 0
  - 6 * 4096 = 24576
- What is the physical address for logical address 20500:
  - Page index 5 offset 20 $\Rightarrow$ frame with index 3 offset 20
  - 3 * 4096 + 20 => 12308

Summary

Take-Home Message

- **Paging** splits logical and physical address spaces into small **pages/frames** to reduce internal and external fragmentation
- **Virtual memory** exploits the principle of **locality** and allows for processes to be **loaded only partially** into memory
- **Large logical address spaces** require “different” approaches